



Temas de Prática de Pesquisa

Uma visão geral dos temas propostos para a disciplina, cobrindo detecção de anomalias, modelos gráficos, predição de atrasos, mobilidade urbana, streaming, clustering, redes neurais e augmentação de dados.

CIÊNCIA DE DADOS

Eduardo Ogasawara

eduardo.ogasawara@cefet-rj.br

<https://eic.cefet-rj.br/~eogasawara>

Previsão de Gols por Concept Drift em Futebol Profissional

Investigar se mudanças estatisticamente detectáveis no padrão de passes antecedem gols em partidas profissionais, utilizando eventos públicos da StatsBomb, detectores de concept drift e avaliação temporal assimétrica para gerar alarmes úteis antes do evento de gol.

Dataset & Contexto

StatsBomb Open Data — La Liga 2015/2016: **380 partidas** e **~1,3 milhão de eventos** com partida, equipe, tempo, tipo de ação, passes, gols e contexto de jogada. Agregação por minuto ou por posse de bola para construção das séries temporais.

ETL Reprodutível

Base temporal padronizada por partida, equipe e período, com exploração de passes, gols open play e placar.

Avaliação SoftED

Alarmes pontuados *apenas antes do gol*, com antecedência-alvo de **10 minutos** e penalização linear assimétrica.

Sinal Temporal & Ferramentas

- Séries temporais de volume de passes por equipe/minuto
- Detectores: **Page-Hinkley**, **KSWIN** e **ADWIN** via pacote *heimdall* (R)
- Suporte: Python, pandas, statsbombpy e validação temporal

Detecção de Drift

Aplicação de Page-Hinkley, KSWIN e ADWIN para identificar mudanças no comportamento ofensivo antes do gol.

Entregáveis

Artigo, código documentado, comparação de detectores, janelas/cooldown e extensões para outras ligas e temporadas.

Mercado Automotivo: Previsão de Vendas

Investigar emplacamentos mensais da Fenabrave e valores da Tabela FIPE para prever vendas por marca, modelo ou segmento, incorporando sazonalidade, histórico de emplacamentos, preço FIPE e variáveis macroeconômicas como juros e PIB.

Sazonalidade

Identificação de padrões mensais e anuais de emplacamentos por marca, modelo e segmento — capturando efeitos de fim de ano, reajustes fiscais e ciclos econômicos.

Variáveis Macroeconômicas

Incorporação de taxa de juros, PIB, câmbio e índices de confiança do consumidor como preditores externos nos modelos de série temporal.

Previsão por Segmento

Modelos específicos por categoria de veículo (hatchback, SUV, pickup, etc.), combustível e faixa de preço FIPE para maior precisão preditiva.

Ranking e Demanda

Análise de participação de mercado e tendência de emplacamentos por modelo, com projeção de ranking mensal e detecção de shifts de demanda.

Dados & Ferramentas

- **Dataset:** Informativos mensais Fenabrave + Tabela FIPE por mês de referência, marca, modelo, ano/modelo e combustível
- **Formato:** Emplacamentos mensais, ranking por modelo/segmento, preço médio FIPE, variação mensal e atributos do veículo
- **Ferramentas:** Python/R, pandas, pdfplumber, rapidfuzz, modelos de séries temporais e regressão

Entregáveis Principais

ETL Padronizado

Pipeline Fenabrave + FIPE com painel mensal integrado e atualizado automaticamente.

Modelo Preditivo

Previsão de emplacamentos por modelo/segmento com sazonalidade e variáveis econômicas.

Análise Exploratória

Sazonalidade, tendências históricas e ranking por modelo e segmento com visualizações interativas.

Avaliação Temporal

Validação cruzada temporal com métricas RMSE e MAPE por horizonte de previsão.

Mercado Automotivo: Previsão de Desvalorização

Estimar a variação mensal e perda acumulada de valor por modelo, versão, idade do veículo e tipo de combustível — identificando padrões de depreciação e construindo curvas de desvalorização com validação temporal cruzada.

Curva de Depreciação

Modelagem da perda acumulada de valor ao longo dos meses/anos por modelo, versão e combustível, com ajuste de curvas de depreciação por segmento.

Fatores de Influência

Análise do impacto de idade, quilometragem estimada, câmbio, juros e lançamentos de novos modelos sobre a taxa de desvalorização.

Sazonalidade da Desvalorização

Identificação de períodos do ano com maior ou menor queda de preço por segmento (ex: fim de ano, lançamentos, reajustes FIPE).

Comparação entre Modelos

Ranking de modelos por taxa de depreciação acumulada em 12, 24 e 36 meses — identificando os que melhor retêm valor.

Dados & Ferramentas

- **Dataset:** Tabela FIPE histórica por mês de referência, marca, modelo, ano/modelo e combustível
- **Formato:** Preço médio mensal por versão, variação percentual mês a mês, atributos do veículo
- **Ferramentas:** Python/R, pandas, modelos de séries temporais, regressão não-linear, visualizações

Entregáveis Principais

ETL FIPE

Pipeline de coleta e padronização histórica da tabela FIPE por versão e mês.

Curvas de Depreciação

Modelos ajustados por segmento, combustível e faixa de preço.

Análise de Fatores

Impacto de variáveis externas (juros, câmbio, lançamentos) na depreciação.

Ranking de Retenção de Valor

Score comparativo entre modelos por horizonte de 12/24/36 meses.

Mercado Automotivo: Oportunidade de Compra e Venda de Semi-novos

Construir um indicador de oportunidade que combina preço relativo, tendência de depreciação, sazonalidade e demanda observada para identificar janelas favoráveis de compra ou venda de veículos semi-novos.

Momento de Venda

Identificação de períodos em que a queda esperada de preço e a liquidez sugerem antecipar a venda de um semi-novo no mercado.

Momento de Compra

Detecção de janelas em que o preço relativo e a tendência de depreciação indicam oportunidade de aquisição abaixo do valor justo.

Score de Oportunidade

Indicador composto por preço relativo ao histórico FIPE, tendência de depreciação, sazonalidade e volume de demanda observada.

Previsão de Preço Futuro

Estimativa do valor esperado do veículo em 3, 6 e 12 meses para apoiar decisões de compra e venda com horizonte definido.

Dados & Ferramentas

- **Dataset:** Tabela FIPE histórica + dados de demanda (anúncios, emplacamentos Fenabreve) por modelo e região
- **Formato:** Preço médio mensal, variação percentual, volume de anúncios, sazonalidade histórica
- **Ferramentas:** Python/R, pandas, modelos de séries temporais, scoring multicritério, dashboards

Entregáveis Principais

ETL Integrado

Pipeline combinando FIPE histórica e sinais de demanda por modelo e região.

Score de Oportunidade

Indicador composto de compra/venda por modelo, versão e período.

Modelo de Preço Futuro

Previsão do valor esperado em 3/6/12 meses com intervalo de confiança.

Dashboard Interativo

Painel para consulta de oportunidades por modelo, faixa de preço e região.

Simulador de Estratégias de Compra e Venda de Ações – IBrX-100

Construir um simulador de backtesting para estratégias de compra e venda de ações das empresas do IBrX-100, usando histórico diário de preços obtido via Yahoo Finance. O objetivo é implementar, comparar e avaliar diferentes estratégias de trading — desde regras clássicas baseadas em indicadores técnicos até abordagens orientadas por modelos preditivos — em um ambiente controlado com custos de transação realistas.

Coleta e Preparação dos Dados

Obter histórico diário (Open, High, Low, Close, Volume, Adjusted Close) das ~100 empresas do IBrX-100 via yfinance (Python). Período sugerido: 2015–2025. Tratar splits, dividendos (usar preço ajustado), dias sem negociação e dados faltantes. Construir painel de preços ajustados com alinhamento temporal entre ativos.

Motor de Backtesting

Simular execução das ordens dia a dia com: custo de transação de 0.1% por operação (corretagem + emolumentos), slippage de 0.05%, capital inicial configurável (ex: R\$ 100.000), posição única por ativo (sem alavancagem), rebalanceamento mensal opcional para estratégias de portfólio.

Dados & Ferramentas

- **Dataset:** IBrX-100 — lista de ativos via B3; histórico diário via yfinance (Python) ou quantmod (R)
- **Período:** 2015–2025 (inclui COVID-19, ciclos de juros, eleições)
- **Ferramentas:** Python, yfinance, pandas, numpy, matplotlib/plotly para visualização
- **Indicadores técnicos:** ta-lib ou implementação manual (SMA, EMA, RSI, Bollinger Bands, MACD)
- **Referência:** Lopez de Prado (2018) — Advances in Financial Machine Learning

Estratégias de Trading Implementadas

(1) Buy & Hold — baseline passivo; (2) Médias Móveis — cruzamento SMA(20) × SMA(50), sinal de compra quando SMA curta cruza SMA longa para cima; (3) RSI — compra quando $RSI < 30$ (sobrevendido), venda quando $RSI > 70$ (sobrecomprado); (4) Bandas de Bollinger — compra na banda inferior, venda na banda superior; (5) Estratégia orientada por modelo preditivo — usar previsão de retorno (LSTM ou XGBoost) como sinal de entrada/saída.

Métricas de Avaliação

Retorno acumulado vs. Buy & Hold e vs. IBrX-100 como benchmark; Sharpe Ratio (retorno ajustado ao risco); Maximum Drawdown (maior queda do pico ao vale); Win Rate (% de operações lucrativas); Calmar Ratio (retorno / max drawdown). Análise por período (bull market, bear market, alta volatilidade).

Entregáveis

- Pipeline de coleta e padronização do histórico diário do IBrX-100 via Yahoo Finance
- Implementação das estratégias clássicas (Buy & Hold, SMA, RSI, Bollinger)
- Motor de backtesting com custos de transação e slippage realistas
- Estratégia orientada por modelo preditivo (LSTM ou XGBoost) como sinal de trading
- Dashboard comparativo de estratégias: retorno acumulado, Sharpe, Drawdown
- Relatório final (12 páginas) + código documentado e reproduzível

Mobilidade Urbana – Congestionamentos

Investigar trajetórias realizadas por ônibus no Rio de Janeiro para **identificar eventos de congestionamento, avaliar efeitos de horário e região e analisar condições climáticas como chuva**, combinando análise espaço-temporal e modelos de clustering sobre dados GPS reais.

Dados e Ferramentas

- **Dataset:** Registros GPS de ônibus do Rio de Janeiro — trajetórias agrupadas em Parquet/RData
- **Formato:** latitude, longitude, timestamp, velocidade, linha e veículo
- **Ferramentas:** R, Python, pandas, geopandas, folium, Haversine
- **Gabarito:** GTFS para comparação posterior com rotas oficiais

Congestionamentos

Agrupar eventos de baixa velocidade e avaliar efeitos de horário, região e condições climáticas como chuva.

Análise Temporal

Investigar a distribuição dos congestionamentos ao longo do dia e da semana, identificando horários de pico, janelas críticas e variações sazonais no fluxo de ônibus.

Entregáveis Esperados

- ETL comum e base padronizada para experimentos
- Identificação de congestionamentos em regiões críticas
- Análise de efeitos de horário, região e condições climáticas (chuva)
- Visualização de mapas de calor de congestionamento

Clustering de Eventos

Aplicar algoritmos de clustering (DBSCAN, K-means) sobre eventos de baixa velocidade para identificar zonas de congestionamento recorrentes e seus padrões espaciais.

Análise Espacial

Mapear regiões críticas da cidade com maior incidência de congestionamentos, cruzando dados de densidade viária, corredores de ônibus e condições climáticas registradas.

Mobilidade Urbana – Pontos Finais e Sentido de Circulação

Investigar trajetórias realizadas por ônibus no Rio de Janeiro para localizar pontos finais (terminais) das linhas e estimar o sentido de circulação (ida/volta), utilizando concentração espacial dos trajetos, produto escalar de deslocamento e segmentação temporal sobre dados GPS reais.

Dados e Ferramentas

- **Dataset:** Registros GPS de ônibus do Rio de Janeiro — trajetórias agrupadas em Parquet/RData
- **Formato:** latitude, longitude, timestamp, velocidade, linha e veículo
- **Ferramentas:** R, Python, pandas, geopandas, folium, Haversine, PaLMTo-Gen
- **Gabarito:** GTFS para comparação posterior com rotas oficiais

Pontos Finais

Localizar terminais prováveis das linhas a partir da concentração e extremidade dos trajetos registrados.

Segmentação de Viagens

Dividir os registros GPS em viagens individuais, identificando início e fim de cada percurso com base em paradas prolongadas e mudanças de direção.

Entregáveis Esperados

- ETL comum e base padronizada para experimentos
- Localização de pontos finais (terminais de início e fim de linha)
- Estimativa de sentido de circulação (ida/volta) por viagem
- Validação dos resultados contra o GTFS oficial

Sentido de Circulação

Estimar ida ou volta por proximidade aos pontos finais, produto escalar de deslocamento e segmentação temporal.

Validação com GTFS

Comparar os pontos finais e sentidos inferidos com as rotas oficiais do GTFS para medir a acurácia do método proposto.

Mobilidade Urbana – Baldeação e GTFS

Inferir conexões de baldeação entre linhas de ônibus no Rio de Janeiro a partir das trajetórias GPS realizadas, confrontando os resultados com o GTFS como gabarito das rotas oficiais — combinando modelos de grafos, análise espaço-temporal e métricas de similaridade de rotas.

Dados e Ferramentas

- **Dataset:** Registros GPS de ônibus do Rio de Janeiro — trajetórias agrupadas em Parquet/RData
- **Formato:** latitude, longitude, timestamp, velocidade, linha e veículo
- **Ferramentas:** R, Python, pandas, geopandas, grafos inferidos, PaLMTo-Gen
- **Gabarito:** GTFS para comparação com rotas e conexões oficiais

Baldeação e GTFS

Inferir conexões pelas trajetórias realizadas e confrontar com o GTFS como gabarito das rotas oficiais.

Deteção de Baldeações

Identificar pontos onde passageiros provavelmente realizam transferência entre linhas, com base em paradas coincidentes e sobreposição de trajetos.

Entregáveis Esperados

- ETL comum e base padronizada para experimentos
- Inferência de pontos de baldeação entre linhas
- Grafo de conexões entre linhas de ônibus
- Validação das baldeações inferidas contra o GTFS oficial

Grafo de Conexões

Construir um grafo onde nós representam linhas de ônibus e arestas representam pontos de baldeação inferidos a partir da proximidade espaço-temporal das trajetórias.

Métricas de Validação

Calcular precisão e recall das baldeações inferidas em relação ao GTFS, avaliando a qualidade do método de inferência proposto.

Integração scikit-learn no DAL Toolbox – Preparação de Dados

Integrar o ecossistema de preparação de dados do scikit-learn ao DAL Toolbox, cobrindo imputação, escalonamento, codificação, seleção de features, pipelines e transformadores compostos — permitindo pré-processamento padronizado e reprodutível para séries temporais e dados tabulares.

Imputação

(sklearn.impute): SimpleImputer, KNNImputer, IterativeImputer — tratamento de valores ausentes por estratégia estatística ou modelo.

Escalonamento

(sklearn.preprocessing): StandardScaler, MinMaxScaler, RobustScaler, Normalizer — normalização e padronização de features numéricas.

Codificação

(sklearn.preprocessing): OneHotEncoder, OrdinalEncoder, LabelEncoder, TargetEncoder — transformação de variáveis categóricas.

Seleção de Features

(sklearn.feature_selection): SelectKBest, RFE, VarianceThreshold, SelectFromModel — redução de dimensionalidade supervisionada e não supervisionada.

Pipeline & ColumnTransformer

Composição de transformações em sequência (Pipeline) e aplicação seletiva por tipo de coluna (ColumnTransformer) — garantindo reprodutibilidade e evitando data leakage.

Transformações Customizadas

FunctionTransformer e BaseEstimator/TransformerMixin para criar transformadores compatíveis com a API sklearn e integráveis ao DAL Toolbox.

Dados & Ferramentas

- **DAL Toolbox DP:** <https://github.com/cefet-rj-dal/daltoolboxdp> (repositório base para integração)
- **Ferramentas:** Python, scikit-learn, pandas, numpy
- **Referência de API:** sklearn.preprocessing, sklearn.impute, sklearn.feature_selection, sklearn.pipeline

Entregáveis

- Wrappers padronizados para cada transformador (imputação, escalonamento, codificação)
- Integração de Pipeline e ColumnTransformer ao fluxo do DAL Toolbox
- Transformadores customizados compatíveis com a API sklearn
- Comparação de estratégias de pré-processamento no dataset de aviação
- Documentação e testes de integração com exemplos reprodutíveis

Integração scikit-learn no DAL Toolbox – Clustering

Integrar métodos de **agrupamento (clustering)** do scikit-learn ao DAL Toolbox, padronizando entrada/saída e adaptando APIs diferentes para uma interface integrada. O valor está em permitir comparação justa entre algoritmos dentro do toolbox, usando o histórico de aviação como dataset.

Dados e Ferramentas

- **DAL Toolbox DP:** <https://github.com/cefet-rj-dal/daltoolboxdp> (repositório base para integração)
- **Ferramentas:** Python, scikit-learn, pandas
- **Referência de API:** [sklearn.cluster](#) (KMeans, DBSCAN, AgglomerativeClustering, OPTICS, Birch, AffinityPropagation, HDBSCAN, etc.)

Entregáveis

- Wrappers padronizados para todos os algoritmos de clustering: KMeans, MiniBatchKMeans, DBSCAN, AgglomerativeClustering, MeanShift, SpectralClustering, GMM, OPTICS, Birch, AffinityPropagation, HDBSCAN
- Comparação de resultados entre algoritmos no dataset de aviação
- **Métricas:** Silhouette Score, Davies-Bouldin Index
- **Referências sklearn:** sklearn.cluster.OPTICS, sklearn.cluster.Birch, sklearn.cluster.AffinityPropagation, sklearn.cluster.HDBSCAN (sklearn ≥ 1.3)

KMeans

Agrupamento por centróides.

MiniBatchKMeans

Versão escalável do KMeans.

DBSCAN

Agrupamento por densidade.

AgglomerativeClustering

Hierárquico aglomerativo.

MeanShift + SpectralClustering + GMM

Métodos adicionais para comparação abrangente.

OPTICS

Variante do DBSCAN com epsilon adaptativo — detecta clusters de densidades variáveis.

Birch

Hierárquico incremental via CF-Tree — eficiente para grandes volumes de dados.

AffinityPropagation

Troca de mensagens entre pontos — número de clusters emergente, sem k fixo.

HDBSCAN

Extensão hierárquica do DBSCAN (sklearn ≥ 1.3) — robusto a ruído e densidades variáveis.

Compressão de Séries Temporais com Autoencoders e Mini Foundation Models

Investigar pipelines de compressão com perdas e erro limitado para séries temporais. A proposta central usa autoencoders para comprimir janelas da série em representações latentes de menor dimensão, comparando com uma baseline de quantização por epsilon + compressão gzip. A evolução futura explora mini modelos especializados por domínio como reconstrutores mais eficientes.

Baseline (Quantização + Gzip)

Quantizar a série com tolerância epsilon (arredondar valores ao múltiplo de ϵ mais próximo) e comprimir com gzip. Simples, interpretável e sem custo de modelo. Serve como referência mínima de taxa de compressão e erro garantido.

Autoencoder por Janelas

Dividir a série em janelas deslizantes de tamanho w . Encoder comprime cada janela em vetor latente z de dimensão $d_z \ll w$. Decoder reconstrói a janela a partir de z . Armazenar vetor latente + resíduo para garantir erro limitado. Avaliar se o ganho de compressão compensa o overhead do modelo e metadados.

Mini Modelos por Domínio

Substituir um único autoencoder genérico por pequenos modelos especializados para domínios como energia, clima ou transportes. Cada mini modelo é treinado no padrão típico do seu domínio, atuando como reconstrutor mais preciso com menor taxa de resíduo.

Seleção Automática de Modelo

Extensão futura: selecionar o mini modelo mais adequado por janela da série com base em metacaracterísticas (variância, tendência, sazonalidade, autocorrelação). Roteamento automático sem intervenção manual.

Dados & Ferramentas

- **Dataset:** Série univariada sintética ou real (ex: ETT-small, M4, ou série de aviação do DAL Toolbox)
- **Ferramentas:** Python/R, PyTorch (autoencoder), numpy, pandas, gzip (baseline)
- **DAL Toolbox:** [daltoolboxdp](#) para encode-decode com autoencoder
- **Parâmetros:** tamanho de janela $w \in \{32, 64, 128\}$, dimensão latente $d_z \in \{4, 8, 16\}$, epsilon $\in \{0.01, 0.05, 0.1\}$

Entregáveis

1. EDA da série temporal escolhida (estacionariedade, sazonalidade, autocorrelação)
2. Baseline: quantização por epsilon + gzip com diferentes valores de epsilon
3. Autoencoder por janelas com armazenamento de resíduo para erro limitado
4. Comparação quantitativa: taxa de compressão, MAE/RMSE de reconstrução, overhead de metadados
5. Discussão sobre viabilidade de mini modelos por domínio como direção futura
6. Relatório final (12 páginas) + código documentado

Predição Exógena: SARX e Encoder-Decoder Neural

Incorporar variáveis exógenas (temperatura, feriados, hora do dia) na previsão de séries temporais — comparando dois paradigmas: o modelo estatístico SARX (extensão do SARIMA com regressores externos) e uma arquitetura neural Encoder-Decoder baseada em LSTM que processa histórico endógeno e exógeno separadamente.

Modelo SARX

Extensão do SARIMA com variáveis exógenas defasadas. Equação: $y_t = \sum \phi_i y_{t-i} + \sum \Phi_s y_{t-s} + \sum \beta_j x^{exo}_{t-j} + \epsilon_t$. Onde ϕ são os lags AR, Φ os lags sazonais (ex: {24,48,168} para dados horários) e β os lags exógenos. Implementação via statsmodels SARIMAX. Seleção de ordem por AIC/BIC com grid search em $p \in \{1..5\}$, $q \in \{0..3\}$.

Variáveis Exógenas Típicas

Temperatura (contínua), dia da semana (one-hot, 7 dimensões), feriados (binário), hora do dia (codificação seno/cosseno para capturar periodicidade circular: $\sin(2\pi \cdot h/24)$, $\cos(2\pi \cdot h/24)$).

Dados & Ferramentas

- Dataset:** Série horária com variáveis exógenas disponíveis (ex: ETT-small com temperatura, ou dados de energia elétrica com temperatura e calendário)
- Ferramentas:** Python, statsmodels (SARIMAX), PyTorch (LSTM Encoder-Decoder), pandas, numpy
- Referência:** Hyndman & Athanasopoulos (2021) — Forecasting: Principles and Practice; Lim & Zohren (2021) — Time-series forecasting with deep learning

Encoder-Decoder Neural (LSTM)

Encoder processa [série histórica + variáveis exógenas históricas] → vetor de contexto c . Decoder usa c + variáveis exógenas futuras → previsão do horizonte H . Permite usar variáveis exógenas conhecidas no futuro (ex: previsão do tempo, calendário de feriados) para melhorar a previsão.

Ablação Obrigatória

Três condições experimentais: **(1) apenas endógena** — baseline sem variáveis externas; **(2) endógena + exógena** — modelo completo; **(3) exógena isolada** — para medir contribuição individual. Métrica principal: ganho de RMSE da configuração completa vs. baseline endógena.

Entregáveis

- Implementação do SARX via SARIMAX com seleção de ordem por AIC/BIC
- Implementação do Encoder-Decoder LSTM com suporte a variáveis exógenas futuras
- Engenharia de features exógenas: one-hot, seno/cosseno, variáveis binárias
- Ablação completa: endógena vs. endógena+exógena vs. exógena isolada
- Comparação SARX vs. Encoder-Decoder em RMSE e MAE por horizonte H
- Relatório final (12 páginas) + código documentado

Vanilla Transformer para Séries Temporais

Entrada

Janela deslizante + z-score local por canal (RevIN)

Embeddings

patch_size P=16, Linear(P·d, d_model=128),
Positional Encoding aprendível

Encoder Blocks

L=3 camadas: MHA (nhead=8) → Residual →
LayerNorm → FFN(dim_ff=512)

Saída

mean(Z) → Linear(d_model, H) ou Z[-1] → Linear

Arquitetura e Código

Objetivo: $f_\theta : \mathbb{R}^{(B \times T \times d)} \rightarrow \mathbb{R}^{(B \times H)}$

d_k = $d_{model}/n_{head} = 16$; **Atenção:** $softmax(QK^T/\sqrt{d_k})V$; Pre-LN mais estável.

Treinamento: Adam lr=1e-4, weight_decay=1e-5, batch=64, epochs=100, CosineAnnealingLR, gradient clipping=1.0.

Complexidade: $O(T^2)$ — para T>512 usar sparse attention ou patches maiores.

```
import torch.nn as nn

class TSTransformer(nn.Module):
    def __init__(self, d_in, d_model=128, nhead=8, num_layers=3, dim_ff=512, H=24):
        super().__init__()
        self.patch_embed = nn.Linear(d_in, d_model)
        self.pos_enc = LearnablePositionalEncoding(d_model)
        encoder_layer = nn.TransformerEncoderLayer(d_model, nhead, dim_ff, dropout=0.1, batch_first=True)
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers)
        self.head = nn.Linear(d_model, H)
    def forward(self, x): # x: (B, T, d_in)
        x = self.patch_embed(x)
        x = self.pos_enc(x)
        z = self.encoder(x)
        return self.head(z.mean(dim=1))
```

Avaliação e Entregáveis

Divisão temporal: train 70%, val 15%, test 15% — NUNCA embaralhar.

Métricas: RMSE, MAE, MAPE por horizonte $H \in \{24, 48, 96\}$.

Baseline naïve: $\hat{y}_{t+h} = y_t$ (persistência) — modelo deve superar.

Teste estatístico: Diebold-Mariano para comparar modelos.

Tabela de comparação (inline, simples):

- LSTM: $O(T)$ / Dep. Longa Média / Estabilidade Alta
- GRU: $O(T)$ / Média / Alta
- Transformer: $O(T^2)$ / Alta / Variável

Entregáveis: código modular (data_loader.py, model.py, train.py, evaluate.py); reprodutibilidade (seed=42); relatório com curva loss, tabela de métricas, análise de erros, gráfico predição vs real.

Checklist de Implementação

- Verificar shapes com `print(x.shape)`
- Confirmar padding mask
- Salvar checkpoints por epoch (best val_loss)
- Testar com T=64, d=1, H=1 antes de escalar
- Monitorar attention weights
- Logging com wandb

xLSTM – LSTM Estendido

1. Arquitetura xLSTM

O xLSTM introduz dois blocos principais que atuam em diferentes tipos de memória para superar as limitações dos LSTMs tradicionais:

- **sLSTM (scalar LSTM):** Utiliza uma memória escalar (vetorial) e incorpora mecanismos de forget gate exponencial e normalização interna para lidar melhor com o gradiente e a retenção de informações.
- **mLSTM (matrix memory LSTM):** Introduz uma memória matricial que permite interações mais ricas entre os estados, crucial para capturar dependências complexas.

Detalhes do sLSTM:

No sLSTM, as equações dos gates e da atualização de estado são modificadas:

$$f_t = \exp(w_f \cdot h_{t-1} + b_f)$$

$$i_t = \exp(w_i \cdot h_{t-1} + b_i)$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot z_t$$

$$n_t = f_t \cdot n_{t-1} + i_t$$

$$h_t = o_t \cdot (c_t / \max(n_t, 1))$$

Onde f_t e i_t são os forget e input gates exponenciais, c_t é o estado da célula, n_t é um normalizador cumulativo, e h_t é o estado oculto. O uso de funções exponenciais e normalização interna melhora a estabilidade numérica e a capacidade de aprender dependências de longo prazo.

Detalhes do mLSTM:

O mLSTM opera com uma memória matricial $C_t \in \mathbb{R}^{(d \times d)}$, permitindo modelar relações mais complexas. As equações são:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot (v_t \cdot k_t^T)$$

$$h_t = o_t \cdot (C_t \cdot q_t / \max(\|C_t \cdot q_t\|, 1))$$

Aqui, v_t e k_t são vetores que formam uma atualização matricial, e q_t é um vetor de query que interage com a memória C_t para produzir o estado oculto h_t . A normalização é aplicada à norma do vetor resultante da multiplicação da matriz.

Empilhamento de Blocos:

A arquitetura xLSTM tipicamente empilha blocos sLSTM e mLSTM de forma alternada para combinar os benefícios de ambos. Uma configuração recomendada é `[mLSTM, sLSTM, mLSTM]` com `d_model=256` para um bom equilíbrio entre capacidade e eficiência.

2. Hiperparâmetros e Avaliação

Para um treinamento e avaliação robustos do xLSTM:

- **Hiperparâmetros:**
 - `d_model=256` (dimensão do modelo)
 - `num_layers=3` (número de camadas empilhadas)
 - `dropout=0.1`
 - `lr=3e-4` (taxa de aprendizado)
 - `batch_size=32`
 - `seq_len=512` (comprimento da sequência de entrada)
- **Estabilidade de Treinamento:**
 - **Gradient clipping:** `max_norm=1.0` é crucial para prevenir gradientes explosivos, especialmente com as operações exponenciais no sLSTM.
- **Avaliação:**
 - **Métricas de Desempenho:** RMSE (Root Mean Squared Error) para previsão de múltiplos horizontes (`H=96, 192, 336, 720`).
 - **Comparação:** Comparar o desempenho com modelos base como LSTM padrão e Transformer para demonstrar os ganhos do xLSTM.
 - **Medição de Estabilidade:** Monitorar a norma do gradiente ao longo do treinamento para verificar a estabilidade do processo de otimização.

TSFM – Modelo Fundacional Pequeno

1. Arquitetura

Encoder-only Transformer com patch embedding para processar séries temporais.

- Input shape: (B, T, d)
- Processamento: patches de tamanho P $\rightarrow$ (B, T/P, d_model)
- Parâmetros: d_model=128, nhead=4, num_layers=3, dim_feedforward=256, dropout=0.1

2. Pré-treino – Masking

Estratégia auto-supervisionada para aprender representações robustas:

- mask(x_t): selecionar 30% dos patches aleatoriamente e zerá-los.
- Substituir patches mascarados por um token [MASK] aprendível.
- Função de Loss: MSE na reconstrução dos patches mascarados.

3. Pré-treino – Segment Swap

Outra estratégia de auto-supervisão:

- Dividir a série temporal em 2 segmentos.
- Trocar os segmentos com probabilidade p=0.5.
- O modelo deve prever se houve swap (tarefa de classificação binária).
- Loss combinada: $L = L_{\text{recon}} + \lambda \cdot L_{\text{swap}}$, onde $\lambda=0.5$.

7. Avaliação

Métricas e comparações para garantir a robustez das soluções:

- Predição: MAE/RMSE por horizonte.
- Detecção: F1-score.
- Comparar o desempenho com modelos treinados do zero.

4. Fine-tuning para Predição

Adaptação para previsão multi-horizonte:

- Descongelar as últimas 2 camadas do encoder.
- Adicionar um MLP head (d_model $\rightarrow$ H, onde H é o horizonte de predição).
- Hiperparâmetros: lr=1e-4, epochs=50, early stopping patience=10.

5. Fine-tuning para Detecção de Eventos

Adaptação para identificação de anomalias ou eventos:

- Adicionar um classification head.
- Estratégias:
 - a. full fine-tune (treinar todas as camadas)
 - b. freeze encoder + treinar só head
 - c. LoRA nos attention layers (rank=4)

6. Pseudocódigo Simplificado

```
model = TSFMEncoder(d_model=128, nhead=4, layers=3)
# Pré-treino
for batch in pretrain_loader:
    x_masked, mask = apply_masking(batch, ratio=0.3)
    x_hat = model(x_masked)
    loss = MSE(x_hat[mask], batch[mask])
# Fine-tune
model.freeze_encoder(layers=[0,1])
model.add_head('forecast', horizon=H)
for batch in finetune_loader:
    y_hat = model(batch['x'])
    loss = MSE(y_hat, batch['y'])
```

Graph Models – GCN + LSTM com ETT-small

Construir um protótipo simplificado de detecção/predição com **GCN + LSTM** para séries temporais multivariadas, usando o dataset ETT-small (temperatura do óleo e carga de dois transformadores, ~2 anos de dados). O foco está no modelo, não em engenharia de dados complexa.

Dataset ETT-small

Disponível no GitHub (ETDataset) e no Kaggle. Inclui temperatura do óleo e carga de transformadores.

<https://github.com/zhouhaoyi/ETDataset>

Buscar "ETT-small" no Kaggle

Arquivos: ETTh1.csv ou ETTm1.csv

Referência Principal

Tutorial Keras "Traffic forecasting using GCN-LSTM" como base prática para o pipeline grafo-temporal.

Tutorial Keras:

https://keras.io/examples/timeseries/timeseries_traffic_forecasting/

Bibliotecas: StellarGraph (instalação: pip install stellargraph)

PyTorch Geometric: <https://pytorch-geometric.readthedocs.io>

Métricas de avaliação: MAE e RMSE (implementações em sklearn.metrics ou torch)

Análise Exploratória

Carregar CSV, visualizar séries e normalizar os dados.

Revisão e Planejamento

Revisão bibliográfica e plano metodológico baseado no tutorial Keras.

Construção do Grafo e Modelo

Cada transformador vira um nó; arestas completas ou por correlação. Pipeline GCN → LSTM com treino, validação e ajuste de hiperparâmetros. Bibliotecas: StellarGraph ou PyTorch Geometric.

Avaliação e Relatório

Métricas MAE e RMSE na predição de temperatura do óleo. Comparar versões com e sem GCN. Relatório final (12 páginas) + código + integração opcional com Harbinger/TSPredIT.

Autoencoder com Adaptive Normalization Layer (AN-AE)

Séries não estacionárias têm média e variância variáveis no tempo ($x_t = \mu_t + \sigma_t z_t$). Normalizações fixas assumem parâmetros constantes, gerando **erro estrutural** quando μ_t varia. O objetivo é aprender um baseline adaptativo com filtros multi-escala ponderados por softmax.

Dados e Ferramentas

Datasets de validação: ETT-small (<https://github.com/zhouhaoyi/ETDataset>), M4 Competition (<https://github.com/Mcompetitions/M4-methods>) ou dataset sintético com regime shift

Ferramentas: PyTorch (torch.nn), Python, pandas, numpy

Referência: Adaptive Normalization (Kim et al., 2021 — RevIN: <https://arxiv.org/abs/2106.06842>)

Regularização: Total Variation (TV) + esparsidade

Entregáveis

- Implementação da Adaptive Normalization Layer (ANL) em PyTorch
- Autoencoder completo com Encoder-Decoder + ANL
- Ablações: sem TV, sem esparsidade, com BatchNorm
- Comparação de reconstrução sob regime shift
- Relatório final (12 páginas) + código documentado

Adaptive Normalization Layer (ANL)

Calcula baseline como combinação de convoluções multi-escala; normaliza a série pelo baseline; aplica softclamp para estabilidade.

Encoder-Decoder

Encoder comprime a série normalizada em representação latente z ; Decoder reconstrói a série a partir de z .

Loss e Ablações

Loss = reconstrução + regularização TV (suavidade do baseline) + esparsidade. Ablações: remover TV, esparsidade, robust loss e comparar com BatchNorm.

Resultado esperado: melhor reconstrução sob regime shift em comparação com normalizações fixas.

LSTM + ANL + Seleção de Lags

A escolha manual de defasagens (p fixo) é subótima para séries temporais. A proposta é **aprender uma máscara diferenciável** que selecione automaticamente os lags mais relevantes durante o treinamento.

O pipeline combina duas inovações: a **ANL (Adaptive Normalization Layer)**, que lida com não-estacionariedade ao aprender um baseline adaptativo multi-escala (séries com média e variância variáveis no tempo: $x_t = \mu_t + \sigma_t z_t$), e a **MaskLag**, que aprende quais defasagens são relevantes via máscara diferenciável.

Entrada X

Série com lags candidatos $x_{t-1}, \dots, x_{t-p}$.

MaskLag

Máscara diferenciável $m_i = \text{sigmoid}(w_i)$ aplicada aos lags: $X'_t = m \odot X_t$.
Seleciona automaticamente os lags mais relevantes durante o treinamento.

Dados e Ferramentas

Datasets: ETT-small (<https://github.com/zhouhaoyi/ETDataset>), M4 (<https://github.com/Mcompetitions/M4-methods>) ou séries do DAL Toolbox

Ferramentas: PyTorch (LSTM, sigmoid, L1 regularization), Python, pandas

Referência ANL: Kim et al. (2021) — RevIN (<https://arxiv.org/abs/2106.06842>)

Referência Lag Selection: Lim & Zohren (2021) — Time-series forecasting with deep learning

Baselines: LSTM padrão, N-BEATS (<https://github.com/ElementAI/N-BEATS>), Transformer

ANL (Adaptive Normalization Layer)

Baseline multi-escala por convolução + softmax; normaliza a série pelo baseline adaptativo; softclamp para estabilidade numérica.

LSTM → Output

Loss: MSE + regularização L1 nos pesos da máscara. Comparar com LSTM padrão, N-BEATS e Transformer.

Entregáveis

- Implementação da MaskLag diferenciável em PyTorch
- Pipeline completo: ANL → MaskLag → LSTM
- Comparação com baselines (LSTM, N-BEATS, Transformer) em MAE e RMSE
- Análise dos lags selecionados (interpretabilidade da máscara)
- Relatório final (12 páginas) + código documentado

Modelos Fundacionais + Adaptive Normalization

Modelos fundacionais de séries temporais (ex: **Moirai**, **TimesFM**, **Chronos**) são pré-treinados em distribuições diversas, mas são **sensíveis à escala e estacionariedade** dos dados de entrada. A proposta é inserir uma **ANL (Adaptive Normalization Layer)** — que aprende um baseline adaptativo multi-escala via convoluções ponderadas por softmax — antes da inferência do modelo fundacional, e avaliar o impacto em zero-shot, fine-tuning e estabilidade cross-domain, comparando com z-score estático (RevIN).

Zero-shot com ANL

Avaliar se a normalização adaptativa melhora a previsão zero-shot em séries com regime shift, comparando com z-score estático (RevIN) como baseline.

Velocidade de Fine-tuning

Medir se a ANL reduz o número de épocas necessárias para convergência ao fazer fine-tuning do modelo fundacional em domínios específicos.

Estabilidade Cross-domain

Testar a robustez da ANL em séries de domínios distintos (energia, clima, finanças, tráfego) — verificando se a normalização adaptativa generaliza melhor que o z-score.

Ablações

Comparar ANL completa vs. sem TV regularization vs. sem softclamp vs. z-score estático vs. sem normalização — isolando a contribuição de cada componente.

Dados e Ferramentas

Modelos fundacionais: Moirai (Salesforce), TimesFM (Google), Chronos (Amazon)

Datasets: ETT-small, M4, Weather, Traffic (benchmarks padrão de séries temporais)

Ferramentas: PyTorch, HuggingFace Transformers, Python, pandas

Referência ANL/RevIN: Kim et al. (2021) — <https://arxiv.org/abs/2106.06842>

Referência Foundation Models: Ansari et al. (2024) — Moirai; Das et al. (2024) — TimesFM

Entregáveis

- Implementação da ANL compatível com a API de inferência dos modelos fundacionais
- Experimentos zero-shot: ANL vs. z-score em séries com e sem regime shift
- Experimentos fine-tuning: curvas de convergência por domínio
- Ablações completas isolando cada componente da ANL
- Relatório final (12 páginas) + código documentado e reproduzível

Aumento de Dados com Awareness – Context-Aware Temporal Augmentation

Construir um operador de augmentação **condicional ao contexto temporal** que preserve autocorrelação, tendência e sazonalidade, sem alterar o regime estatístico dominante. Augmentações globais como jittering ou time-warping podem destruir a estrutura de dependência temporal — o objetivo é tornar o operador dependente do contexto local.

Variance-aware Jitter

Ruído proporcional à variância local: $\epsilon_t \sim N(0, \sigma^2_{local})$

Regime-aware Scaling

Escalonamento condicional ao regime identificado: $\tilde{x}_t = \alpha_r \cdot x_t$

Seasonality-aware Time Warping

Deformação temporal aplicada apenas dentro de ciclos sazonais detectados.

Pipeline

- Série de Entrada
- Detecção de Regime (opcional)
- Estatísticas Locais (média, variância, ACF)
- Módulo de Aumento Condicional
- Série Aumentada

Protocolo Experimental

Comparar três condições: sem augmentação, augmentação padrão e augmentação com awareness. Modelos avaliados: LSTM, Transformer e xLSTM. Métricas: RMSE e robustez por condição; Divergência KL entre distribuições original/aumentada; Preservação de ACF e estabilidade espectral. Contribuição científica: definição formal de Context-aware Temporal Augmentation (CTA).

Aprendizado Contínuo: Plasticidade vs Estabilidade

Fluxo de dados $D_1, D_2, \dots, D_T$ — treinar sequencialmente sem esquecer: $\theta_t \approx \theta_{t-1}$.

Replay (Experience Replay)

- Memória M: buffer de tamanho fixo (ex: 500 exemplos). Estratégia de seleção: reservoio sampling (garante representatividade uniforme).
- Reservoir sampling:

```
class ReservoirMemory:
    def __init__(self, capacity=500):
        self.M, self.n = [], 0
    def update(self, x):
        self.n += 1
        if len(self.M) < self.capacity:
            self.M.append(x)
        else:
            j = random.randint(0, self.n)
            if j < self.capacity:
                self.M[j] = x
```

- Treino: $D'_t = D_t \cup \text{sample}(M, k)$. $k = \min(|D_t|, 50)$.
Loss: $\text{MSE}(\text{model}(D'_t), y'_t)$.

EWC (Elastic Weight Consolidation)

- Após treinar em tarefa t: calcular importância F_i (diagonal da matriz de Fisher) para cada parâmetro θ_i .
- $F_i = E[(\partial \log P(y|x, \theta) / \partial \theta_i)^2] \approx$ média dos gradientes² no dataset atual.
- Loss EWC: $L = L_t(\theta) + \lambda/2 \cdot \sum_i F_i \cdot (\theta_i - \theta_i^*)^2$.
 $\lambda=1000$ (ajustar por validação).

```
def ewc_loss(model, fisher, theta_star,
             lambda_=1000):
    penalty = 0
    for n, p in model.named_parameters():
        penalty += (fisher[n] * (p -
            theta_star[n])**2).sum()
    return lambda_ / 2 * penalty
# Calcular Fisher após cada tarefa:
fisher = {n: g**2 for n, g in zip(names, grads)}
```

Expansão Dinâmica

- Detectar novo regime via drift detector (ADWIN ou MMD). Se novo regime: adicionar módulo especializado.
- Arquitetura: backbone compartilhado + heads especializadas por regime. Roteamento: classificador de regime → seleciona head.
- Progressive Neural Networks: coluna nova por tarefa, lateral connections para colunas anteriores (sem modificá-las).
- Avaliação: Backward Transfer (BWT) = $\sum (R_{T,i} - R_{i,i}) / T$. BWT negativo = esquecimento. Forward Transfer (FWT) = ganho em tarefas futuras. Protocolo: treinar sequencialmente em $D_1, \dots, D_T$, avaliar em todos após cada tarefa.

Reconhecimento de Eventos – Matrix Profile em R

Implementar do zero os diferentes algoritmos de Matrix Profile em R, **sem dependência de bibliotecas externas** (como tsmp), para integração ao DAL Toolbox. O Matrix Profile armazena a distância z-normalizada de cada subsequência ao seu vizinho mais próximo — permitindo detectar **discordâncias** (anomalias), **motifs** (padrões recorrentes) e eventos de forma eficiente.

STAMP

Complexidade: $O(n^2 \log n)$

Algoritmo base — para cada subsequência i , calcula a distância z-normalizada a todas as outras usando **MASS** (Mueen's Algorithm for Similarity Search) via FFT. Referência: Yeh et al. (2016).

STOMP

Complexidade: $O(n^2)$

Versão incremental do STAMP — reutiliza o produto escalar da iteração anterior via **atualização sliding**, eliminando FFT por janela. Mais eficiente na prática para séries longas.

SCRIMP++

Complexidade: $O(n^2 / \text{amostragem})$

Versão **anytime** — amostra diagonais aleatórias da matriz de distâncias e refina progressivamente. Retorna MP aproximado rapidamente, melhorando com mais tempo disponível.

Dados & Ferramentas

- **Dataset:** Séries temporais de aviação do DAL Toolbox (daltoolboxdp)
- **Linguagem:** R puro — sem tsmp, sem bibliotecas externas de MP
- **Implementação:** MASS via FFT (stats::fft), produto escalar sliding (STOMP), amostragem diagonal (SCRIMP++)
- **Parâmetros:** comprimento de subsequência m (testar $m \in \{20, 50, 100\}$), exclusion zone = $m/4$
- **Referências:** Yeh et al. (2016) — STAMP; Zhu et al. (2016) — STOMP; Zhu et al. (2018) — SCRIMP++

Detecção de Eventos

Threshold adaptativo

Discordâncias = picos (max do MP); motifs = vales (min do MP). Threshold: $\text{mean}(MP) + k \cdot \text{sd}(MP)$. Aplicação ao dataset de aviação do DAL Toolbox.

Entregáveis

- Implementação em R de **STAMP** com MASS via FFT (função base)
- Implementação em R de **STOMP** com atualização incremental do produto escalar
- Implementação em R de **SCRIMP++** com amostragem diagonal anytime
- Comparação de tempo de execução e precisão entre os três algoritmos
- Detecção de discordâncias e motifs no dataset de aviação com threshold adaptativo
- Relatório final (12 páginas) + código documentado e testes de validação

Drift Multivariado

Formalização

Drift ocorre quando a distribuição muda ao longo do tempo: $P_t(X) \neq P_{t+k}(X)$

Tipos: (a) sudden — mudança abrupta, (b) gradual — transição suave, (c) recorrente — regime anterior retorna.

ADWIN (Adaptive Windowing)

Divisão adaptativa de janela — detecta automaticamente o ponto de mudança sem tamanho fixo de janela.

```
from river.drift import ADWIN
detector = ADWIN(delta=0.002) # delta: confiança
for x_t in stream:
    detector.update(x_t)
    if detector.drift_detected:
        print(f"Drift em t={t}, janela={detector.width}")
        # resetar modelo ou disparar retreino
```

Parâmetros ADWIN: δ = 0.002 (mais sensível: 0.05, menos: 0.0001). Internamente mantém janela W e divide em W_0, W_1 testando $|\mu_0 - \mu_1| > \epsilon_{cut}$.

Para multivariado: aplicar ADWIN por canal + lógica de votação (drift se $\geq k/d$ canais detectam).

Estratégias de Resposta ao Drift

Ao detectar drift, podemos adotar:

1. **Retreino completo:** descartar histórico, treinar com janela recente.
2. **Retreino incremental:** fine-tune com lr reduzido ($lr \times 0.1$) nos últimos w exemplos.
3. **Ensemble com peso temporal:** $w_i \propto \exp(-\lambda \cdot age_i)$, $\lambda = 0.01$.

Detecção multivariada alternativa: MMD

MMD (Maximum Mean Discrepancy) entre janelas é uma outra abordagem.

$$\text{MMD}^2(\mathbf{P}, \mathbf{Q}) = E[k(x, x')] - 2E[k(x, y)] + E[k(y, y')]$$

$$\text{Kernel RBF: } k(x, y) = \exp(-||x - y||^2 / 2\sigma^2).$$

```
def mmd2(X, Y, sigma=1.0):
    XX = rbf_kernel(X, X, sigma)
    YY = rbf_kernel(Y, Y, sigma)
    XY = rbf_kernel(X, Y, sigma)
    return XX.mean() - 2*XY.mean() + YY.mean()
```

Avaliação

- Detecção precoce
- Overhead computacional
- False Positive Rate
- True Positive Rate

Protocolo: injetar drift sintético em posições conhecidas para avaliar a performance do detector.

Lifelong Wasserstein CPD – Detecção de Mudança de Regime

Detectar pontos de mudança (change points) em séries temporais usando a Distância de Wasserstein entre janelas deslizantes — e estender o detector para o cenário lifelong, onde regimes anteriores podem reaparecer e devem ser reconhecidos, não tratados como novidade.

Distância de Wasserstein

Mede a diferença entre duas distribuições empíricas W_1 e W_2 (janelas de tamanho w). Para $p=1$, equivale à área entre as CDFs empíricas. Mais robusta que KL-divergência por não exigir suporte comum.

Fórmula: $W_p(W_1, W_2) = (\int_0^1 |F_1^{-1}(u) - F_2^{-1}(u)|^p du)^{1/p}$

Extensão Lifelong

Ao detectar um change point, o segmento atual é comparado com protótipos de regimes anteriores via Wasserstein.

- Se $W_p(\text{novo}, \text{protótipo}_i) < \epsilon \rightarrow$ regime recorrente (reutilizar modelo)
- Se novo $\rightarrow$ adicionar protótipo ao repositório

Estrutura: lista de (representação do segmento, timestamp, contagem de ocorrências).

Dados & Ferramentas

- **Datasets:** HASC (atividade humana), Yahoo Anomaly Benchmark, UCR Change Point Dataset
- **Ferramentas:** Python, `scipy.stats.wasserstein_distance`, numpy, pandas
- **Drift sintético:** injetar mudanças de média/variância em posições conhecidas para avaliação controlada
- **Referências:** Wasserstein CPD — Staerman et al. (2021); Lifelong learning — Lopez-Paz & Ranzato (2017)

Detecção Online

Janela deslizante de tamanho $2w$ dividida em W_1 (passado) e W_2 (presente). Score = $Wasserstein(W_1, W_2)$.

Threshold adaptativo via EMA: $\tau_t = \mu_t + k \cdot \sigma_t$, com μ e σ atualizados a cada passo. Change point detectado quando score $> \tau_t$.

Parâmetros: $w=50$, $k=3.0$, decay EMA=0.9.

Avaliação

- Latência: $E[t_{\text{detectado}} - t_{\text{real}}]$ em amostras
- F1 com tolerância $\pm w/2$
- Taxa de falsos positivos por hora
- Recall em drift recorrente

Datasets: HASC, Yahoo Anomaly Benchmark, UCR Change Point.

Entregáveis

- Implementação do detector online com threshold adaptativo via EMA
- Extensão lifelong com repositório de protótipos de regime
- Experimentos com drift sintético (sudden, gradual, recorrente) em posições conhecidas
- Comparação com ADWIN e MMD como baselines de detecção
- Relatório final (12 páginas) + código documentado e reproduzível

Lifelong Continual Learning para Anomalias

Detectores de anomalia treinados em um regime perdem desempenho quando o padrão da série muda (**concept drift**). O objetivo é construir um detector que aprenda continuamente — adaptando-se a novos regimes sem esquecer os anteriores — e que reconheça **regimes recorrentes** para reutilizar modelos já treinados, em vez de retreinar do zero.

Detector Base

Qualquer detector com interface `fit(X)` e `score(X) → [0,1]` (ex: autoencoder, LOF, iForest). O score representa o grau de anomalia de cada ponto — quanto maior, mais anômalo. Threshold inicial: $\tau = \text{mean}(\text{score}) + k \cdot \text{std}(\text{score})$ com $k=3.0$.

Replay Seletivo

Ao identificar um regime recorrente, $k=20$ exemplos representativos daquele regime são reapresentados ao detector para evitar esquecimento catastrófico. Seleção por **reservoir sampling** para garantir representatividade.

Dados & Ferramentas

- **Dataset:** Séries sintéticas com regimes controlados ($R_1 \rightarrow R_2 \rightarrow R_1 \rightarrow R_3 \rightarrow R_1$) + Yahoo Anomaly Benchmark
- **Ferramentas:** Python, scikit-learn (LOF, IsolationForest), PyTorch (autoencoder), numpy, pandas
- **Protocolo de drift:** injetar mudanças de média/variância em posições conhecidas para avaliação controlada
- **Referência:** Lopez-Paz & Ranzato (2017) — Continual Learning; Pang et al. (2021) — Deep Anomaly Detection

Memória de Protótipos

Cada regime observado é representado por um protótipo (centroide, covariância, contagem, threshold calibrado). Ao detectar um change point, o segmento atual é comparado via **Distância de Mahalanobis**. Se $\text{dist} < \tau_{\text{proto}}$ → regime conhecido → reutilizar modelo e threshold. Se novo → criar novo protótipo.

Threshold Dinâmico

O threshold é atualizado continuamente via EMA: $\mu_t = \alpha \cdot \text{score}_t + (1-\alpha) \cdot \mu_{t-1}$; $\tau_t = \mu_t + k \cdot \sigma_t$. Parâmetros: $\alpha=0.05$, $k=3.0$. Permite adaptação gradual sem perder sensibilidade.

Entregáveis

- Implementação do detector base com interface padronizada `fit/score`
- Memória de protótipos com identificação de regime via Mahalanobis
- Replay seletivo com reservoir sampling
- Threshold dinâmico via EMA
- Avaliação: F1 em regimes recorrentes, FPR ao longo do tempo, detection delay médio
- Relatório final (12 páginas) + código documentado

Detecção de Anomalias Multivariadas: AE, Cópulas e Grafos

Séries temporais multivariadas apresentam anomalias que podem ser invisíveis em cada canal isolado, mas evidentes nas dependências entre variáveis. O objetivo é comparar três abordagens complementares para detecção de anomalias conjuntas: autoencoders (reconstrução), cópulas (dependência estatística) e grafos de correlação dinâmica (estrutura relacional).

Autoencoder Multivariado

Recebe $X \in \mathbb{R}^{T \times d}$ e aprende a reconstruir a série. Score de anomalia = erro de reconstrução por timestep: $\|X - \hat{X}\|^2$.

Arquitetura: Encoder $\text{Linear}(d \rightarrow h \rightarrow z)$ + Decoder $\text{Linear}(z \rightarrow h \rightarrow d)$, com $z=32$, $h=64$. Extensão VAE: adicionar termo KL à loss para regularizar o espaço latente.

Anomalia = timestep com erro acima do threshold
 $\tau = \text{mean} + k \cdot \text{std}$.

Cópulas (Dependência Conjunta)

Modela a estrutura de dependência entre variáveis separando as marginais da dependência conjunta.

- **Passo 1:** ajustar marginal F_i para cada variável
- **Passo 2:** transformar para espaço uniforme $u_i = F_i(x_i)$
- **Passo 3:** ajustar cópula $C(u_1, \dots, u_d)$ — Gaussiana, Clayton ou Gumbel

Score de anomalia: $-\log C(F_1(x_1), \dots, F_d(x_d))$. Alto score = comportamento conjunto improvável.

Grafos de Correlação Dinâmica

Construir grafo A_t onde $A_t[i, j] = |\text{corr}(X_{t-w:t, i}, X_{t-w:t, j})|$ se $> \theta$, senão 0.

Aplicar GCN simples: $H = \text{ReLU}(D^{-1/2} A D^{-1/2} X W)$.

Anomalia estrutural: comparar A_t com A_{baseline} via norma de Frobenius $\|A_t - A_{\text{baseline}}\|_F > \tau$. Recalcular A a cada $w/2$ passos (sliding com overlap 50%).

Dados & Ferramentas

- **Dataset:** ETT-small (temperatura do óleo e carga de transformadores, ~2 anos, multivariado); SWAT (sistema de tratamento de água com anomalias rotuladas)
- **Ferramentas:** Python, PyTorch (autoencoder, GCN manual com `torch.mm`), `scipy/pingouin` (cópulas), `numpy`, `pandas`
- **Parâmetros:** $z=32$, $h=64$ (AE); $\theta=0.3$, $w=50$ (grafo); $k=3.0$ (threshold)
- **Referência:** Audibert et al. (2020) — USAD; Li et al. (2021) — MTAD-GAT

Entregáveis

- Implementação do autoencoder multivariado com score por timestep
- Implementação de cópula Gaussiana com detecção por log-likelihood conjunta
- Implementação do grafo de correlação dinâmica com GCN manual
- Comparação das três abordagens em F1, Precision e Recall no ETT-small e SWAT
- Análise qualitativa: quais tipos de anomalia cada método detecta melhor
- Relatório final (12 páginas) + código documentado

Próximos Passos

Escolha do Tema

Selecionar o tema de acordo com interesse, habilidades e disponibilidade de dados.

Início da Análise Exploratória

Carregar e explorar o dataset correspondente, produzindo estatísticas e visualizações iniciais.

Entregáveis Progressivos

Seguir o plano de trabalho com apresentações e protótipos em cada etapa, culminando no relatório final de 12 páginas com código documentado.

